Adding fields to a Ninja serializer only requires you to declare them in the class with a corresponding Python type hint. When an object gets serialized, the field gets populated by looking up properties with the same name on the object. To implement the slug and URL fields, you could add `@property` values to `Venue`. Ninja also provides a way of populating fields within the serializer class itself, using a static method named after the field with a `resolve_` prefix. To implement our two fields, you create `.resolve_slug()` and `.resolve_url()`. Figure 11.3 shows how the `.resolve` methods get used by the serializer to produce the resulting JSON.

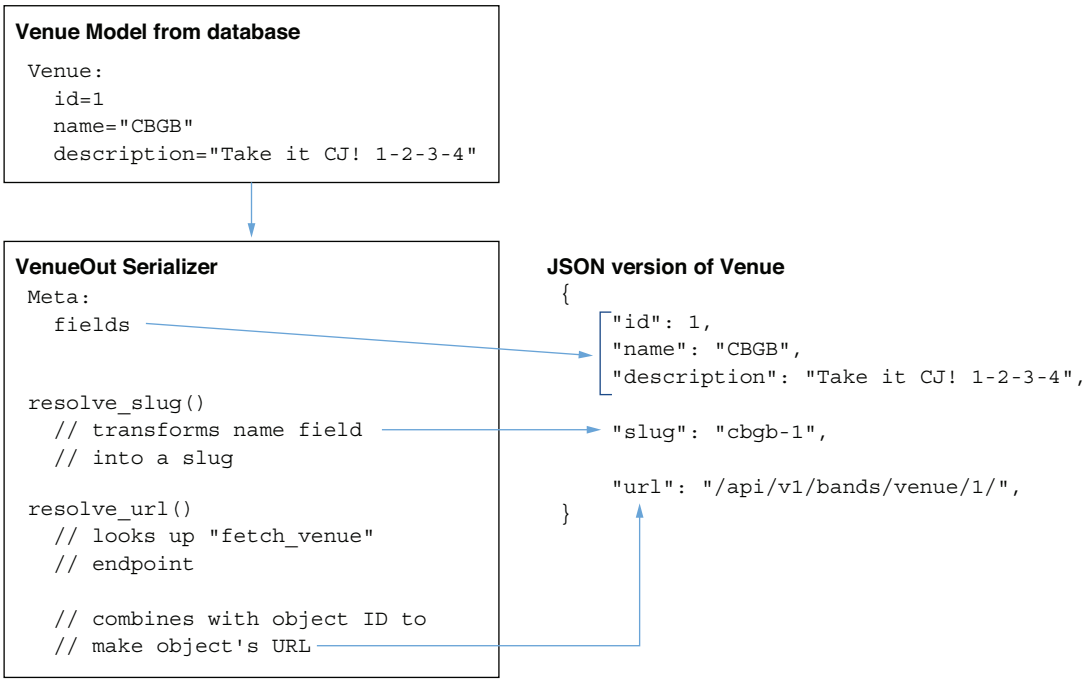


Figure 11.3 Using methods in a Ninja serializer to add content to an object's representation

Django has its roots in the newspaper industry and so comes with a slug method built-in. The `django.utils.text` module has a function called `slugify()`, which takes a string and returns a slug that is URL compatible. The `slugify()` function converts a string to ASCII (in case it contains Unicode); changes it to lowercase; removes leading and trailing spaces; replaces remaining whitespace with a single dash; and removes any characters that aren't alphanumeric, underscores, or hyphens. The `.resolve_slug()` method on the serializer calls `slugify()` on the `Venue` object's name and appends a hyphen and the object ID.

In chapter 5, you learned about the `reverse()` function, which gets used to look up a named URL. The `.resolve_url()` method uses `reverse()` to find the GET endpoint